

Transformation Priority Premise - What is “Obvious implementation” ?

#	TRANSFORMATION	STARTING CODE	FINAL CODE
1	<code>{}</code> => <code>nil</code>		<code>return nil</code>
2	<code>nil</code> => constant	<code>return nil</code>	<code>return "1"</code>
3	<code>constant</code> => constant+	<code>return "1"</code>	<code>return "1" + "2"</code>
4	<code>constant</code> => scalar	<code>return "1" + "2"</code>	<code>return argument</code>
5	<code>statement</code> => statements	<code>return argument</code>	<code>return arguments</code>
6	<code>unconditional</code> => conditional	<code>return arguments</code>	<code>if(condition) return arguments</code>
7	<code>scalar</code> => array	<code>dog</code>	<code>[dog, cat]</code>
8	<code>array</code> => container	<code>[dog, cat]</code>	<code>{dog = "DOG", cat = "CAT"}</code>
9	<code>statement</code> => recursion	<code>a + b</code>	<code>a + recursion</code>
10	<code>conditional</code> => loop	<code>if(condition)</code>	<code>while(condition)</code>
11	<code>recursion</code> => tail recursion	<code>a + recursion</code>	<code>recursion</code>
12	<code>expression</code> => function	<code>today - birthday</code>	<code>CalculateAge()</code>
13	<code>variable</code> => mutation	<code>day</code>	<code>var day = 10; day = 11;</code>
14	switch case		

